IDS Fundamentals

As we know,, firewalls are the security solution usually deployed on the boundary of a network to protect its incoming and outgoing traffic. The firewall checks the traffic when a connection is going to take place and denies it if it violates the firewall rules. However, there should be some security to detect the activities of the connection that passed through the firewall and has already taken place. So, if an attacker successfully bypasses a firewall via a legitimate-looking connection and then performs any malicious activities inside the network, there should be something to detect it in a timely manner. For this purpose, we have a security solution inside the network. This solution is known as an `Intrusion Detection System (IDS)`.

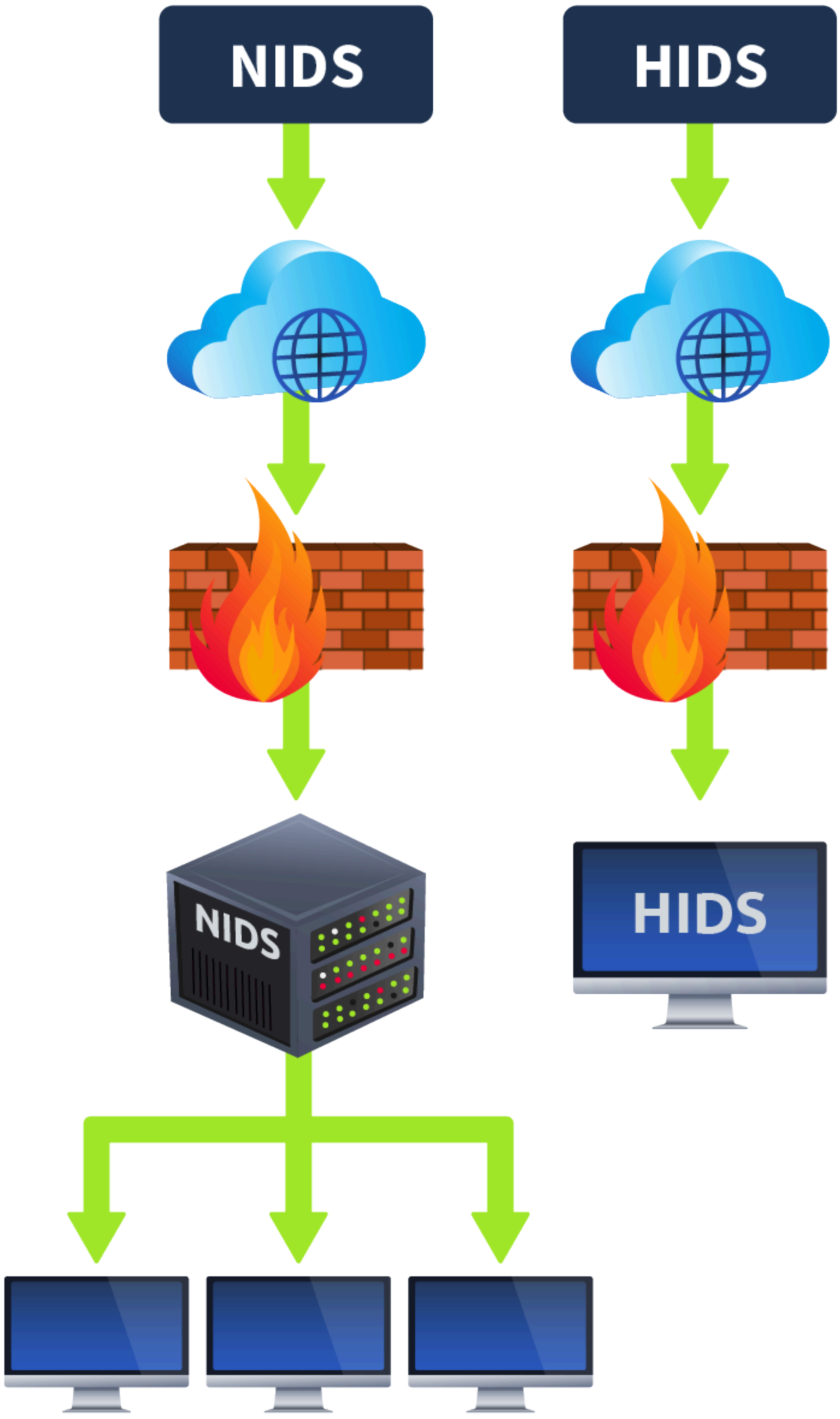
Think of an example of a building's security. A firewall acts as the gatekeeper, checking the people coming in and going out. There is always a chance that some bad actor will successfully sneak inside and start performing malicious activities. He was missed at the gate, but what if we catch him even after he gets in? This can be done by the surveillance cameras present throughout the building. The IDS plays the role of surveillance cameras. It sits in a corner, monitors the network traffic based on its signature and anomaly-based detections, and detects any abnormal traffic going out or inside the network. Upon every detection, an alert is generated for the security administrators. IDS does not act on those detections; it only notifies the security administrators about the malicious activity.

IDS can be categorized differently depending on certain factors. An IDS's main categorization depends on its deployment and detection modes.

Deployment Modes

IDS can be deployed in the following ways:

- **Host Intrusion Detection System (HIDS):** Host-based IDS solutions are installed individually on the hosts and are responsible for only detecting potential security threats associated with that particular host. They provide detailed visibility of the host's activities. However, host intrusion detection systems can be challenging to manage in large networks as they are resource-intensive and require management on each host.
- **Network Intrusion Detection System (NIDS):** Network-based IDS solutions are crucial in detecting potentially malicious activities within the whole network, regardless of any specific hosts. They monitor the network traffic of all the hosts involved to detect suspicious activities. It provides a centralized view of all the detections inside the whole network.



Detection Modes

- **Signature-Based IDS:** Many attacks occur every day. Each attack has its unique pattern, which is known as a signature. These signatures are preserved by the IDS in their databases so that if the same attack happens in the future, it gets detected by its signature and reported to the security administrators for action. The stronger the signature database of the IDS is, the more efficiently it will detect known threats. However, the signature-based IDS is unable to detect zero-day attacks. Zero-day attacks have no prior signatures (patterns) and are not saved inside the IDS databases. Therefore, the signature-based IDS can only detect the attacks that

happened previously, and its signatures (patterns) are saved inside the database. In the upcoming tasks, we will explore a signature-based IDS named Snort.

- **Anomaly-Based IDS:** This type of IDS first learns the normal behavior (baseline) of the network or system and performs detections if there is any deviation from the normal behavior. Anomaly-based IDS can also detect zero-day attacks because they don't rely on available signatures for detection. They detect abnormalities inside the network or system by comparing the current state with the normal behavior (baseline). However, this type of IDS may generate a lot of false positives (marking benign activities as malicious) because the nature of most legitimate programs matches the malicious ones. Anomaly-based IDS would mark them malicious and believe anything behaving unusually is malicious. We can also reduce the false positives generated by anomaly-based IDS by fine-tuning it (manually defining the normal behavior in the IDS).
- **Hybrid IDS:** A hybrid IDS combines the detection methods of signature-based IDS and anomaly-based IDS to leverage the strengths of each approach. Some known threats may already have some signatures in the IDS database; in this case, the hybrid IDS would use the detection technique of the signature-based IDS. If it encounters a new threat, it can leverage the detection method of anomaly-based IDS.

Signature-based IDS can detect threats quickly, while other IDS can have a high processing overhead. However, it is also essential to consider the IDS based on several different factors. Signature-based IDS can be a good option for covering a small threat surface. Anomaly-based IDS and hybrid IDS can help detect modern zero-day attacks, which are increasing daily and can cause massive damage to organizations.

SNORT

Snort is one of the most widely used open-source IDS solutions developed in 1998. It uses signature-based and anomaly-based detections to identify known threats. These are defined in the rule files of the Snort tool. Several built-in rule files come pre-installed in this tool's package. These built-in rule files contain a variety of known attack patterns. Snort's built-in rules can detect a lot of malicious traffic for you. However, you can configure Snort to detect specific types of network traffic not covered by the default rule files. You can create custom rules based on your requirements to detect specific traffic. You can also disable any built-in detection rules if they don't point to harmful traffic for your system or network and define some custom rules instead.

Modes of Snort

Mode	Description	Use Case
Packet sniffer mode	This mode reads and displays network packets without performing any analysis on them. The packet sniffer mode of Snort does not directly relate to IDS capabilities, but it can be helpful in network monitoring and troubleshooting. In some cases, system administrators might need to read the traffic flow without performing any detection to diagnose specific issues. In this case, they can utilize the packet sniffer mode of Snort. This mode allows you to display the network traffic on the console or even output it in a file.	The network team observes some network performance issues. To diagnose the issue, they need detailed insights into the network traffic. For this purpose, they can utilize Snort's packet sniffer mode.

Mode	Description	Use Case
Packet logging mode	Snort performs detection on the network traffic in real-time and displays the detections as alerts on the console for the security administrators to take action. However, in some cases, the network traffic needs to be logged for later analysis. The packet logging mode of Snort allows you to log the network traffic as a PCAP (standard packet capture format) file. This includes all the network traffic and any detections from it. Forensic investigators can use these Snort log files to perform the root cause analysis of previous attacks.	The security team needs to initiate a forensic investigation of a network attack. They would need the traffic logs to perform the root cause analysis. The network traffic logged through Snort's packet logging mode can help them.
Network Intrusion Detection System mode	Snort's NIDS mode is the primary mode that monitors network traffic in real-time and applies its rule files to identify any match to the known attack patterns stored as signatures. If there is a match, it generates an alert. This mode provides the main functionality of an IDS solution.	The security team must proactively monitor their network or systems to detect potential threats. They can leverage Snort's NIDS mode to achieve this.

The most relevant use of Snort as an IDS comes from its NIDS mode. However, Snort can be used in any of the above modes depending upon the requirement.

During Snort installation, you must provide your network interface and range. You can run Snort normally, where it only captures the traffic intended for your host. However, if you want to use Snort to capture and detect intrusions in your whole network, you must turn on the promiscuous mode of your host's network interface.

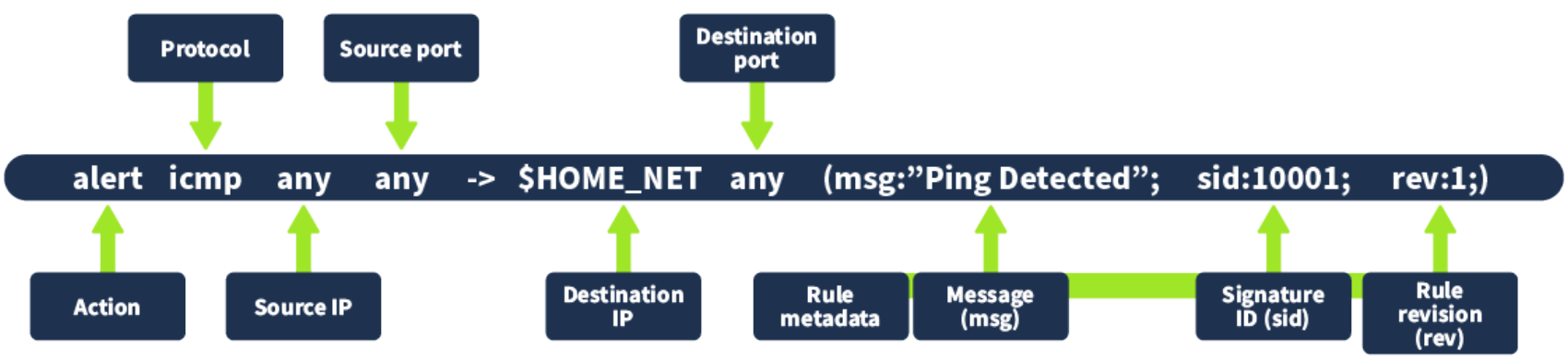
Snort has some built-in rule files, a configuration file, and other files. These are stored in the `/etc/snort` directory. The key file for Snort is its configuration file `snort.conf`, where you can specify which rule files to enable, which network range to monitor, and allow other settings. The rule files are stored in the `rules` folder. Let's use the `ls` command to list all the files and folders present in Snort's main directory:

Snort Directory

```
ubuntu@tryhackme:~$ ls /etc/snort
classification.config  reference.config  snort.debian.conf
community-sid-msg.map  rules             threshold.conf
gen-msg.map            snort.conf        unicode.map
```

Rule Format

Now, let's discuss how rules are created in Snort. There is a specific way of writing the rules. A sample rule would detect ICMP packets (usually used when you ping a host) coming from any IP address and port and reaching the home network (the network range is defined in Snort's configuration file) to any port. Once such traffic is detected, it generates "Ping Detected" alerts.



The details of the components involved in this rule are given below:

- **Action:** This specifies which action to take when the rule triggers. In this case, we have the action to "alert" when the traffic matches this rule.
- **Protocol:** This refers to the protocol that matches this rule. In this case, we use the protocol "ICMP" when pinging a host.
- **Source IP:** This determines the IP originating from the traffic. Since we want to detect traffic from any source IP, we set this as "any".

- **Source port:** This determines the port from which the traffic originates. Since we want to detect traffic from any source port, we set this as "any".
- **Destination IP:** This specifies the destination IP to which the matching traffic comes; it generates the alert. In this case, we used "$HOME_NET". This is a variable, and we defined its value as our whole network's range in Snort's configuration file.
- **Destination port:** This specifies the port the traffic would reach. Since we want to detect traffic coming to any port, we set it to "any."
- **Rule metadata:** Every rule has some metadata. That is defined at the end of the rule in parentheses. The following are its components:
  - **Message (msg):** This describes the message displayed when the subject rule triggers. The message should indicate the type of activity detected. In this case, we used "Ping Detected".
  - **Signature ID (sid):** Every rule has a unique identifier that differentiates it from others. This identifier is called the signature ID (sid). In this case, we set the sid to "10001".
  - **Rule revision (rev):** This sets the rule's revision number. Every time the rule is modified, its revision number is incremented, which helps in tracking changes to any rule.

Rule Creation

Let's paste the sample rule explained above into the custom "local.rules" file in the Snort rules directory.

Firstly, open the "local.rules" file in a text editor:

Edit Custom Rule File

```
ubuntu@tryhackme:~$ sudo nano /etc/snort/rules/local.rules
```

Now, add the following rule after the already present rules to the file:

`alert icmp any any -> 127.0.0.1 any (msg:"Loopback Ping Detected"; sid:10003; rev:1;)`

Note: We will need the other rules in the next task, so do not delete them.

Once you successfully edit the file, press "ctrl+x" and it will ask you to press "y" if you want to save the changes. Press "y" to save the changes.

Rule Testing

Let's first start the Snort tool to detect any intrusions defined in the rule file. For this, we have to execute the following command with sudo privileges in our console:

Running Snort for Detections

```
ubuntu@tryhackme:~$ sudo snort -q -l /var/log/snort -i lo -A console -c /etc/snort/sno
rt.conf
```

Note: In case your loopback interface is not called "lo", replace it with the correct interface name.

As this rule is designed to alert us on any ICMP packets to our loopback address, let's try to ping our loopback address to see if our rule works:

Ping Host

```
ubuntu@tryhackme:~$ ping 127.0.0.1
```

The screenshot below shows the Snort-generated "Loopback Ping Detected" alert when we ping our host's loopback IP. This means that our rule is working fine.

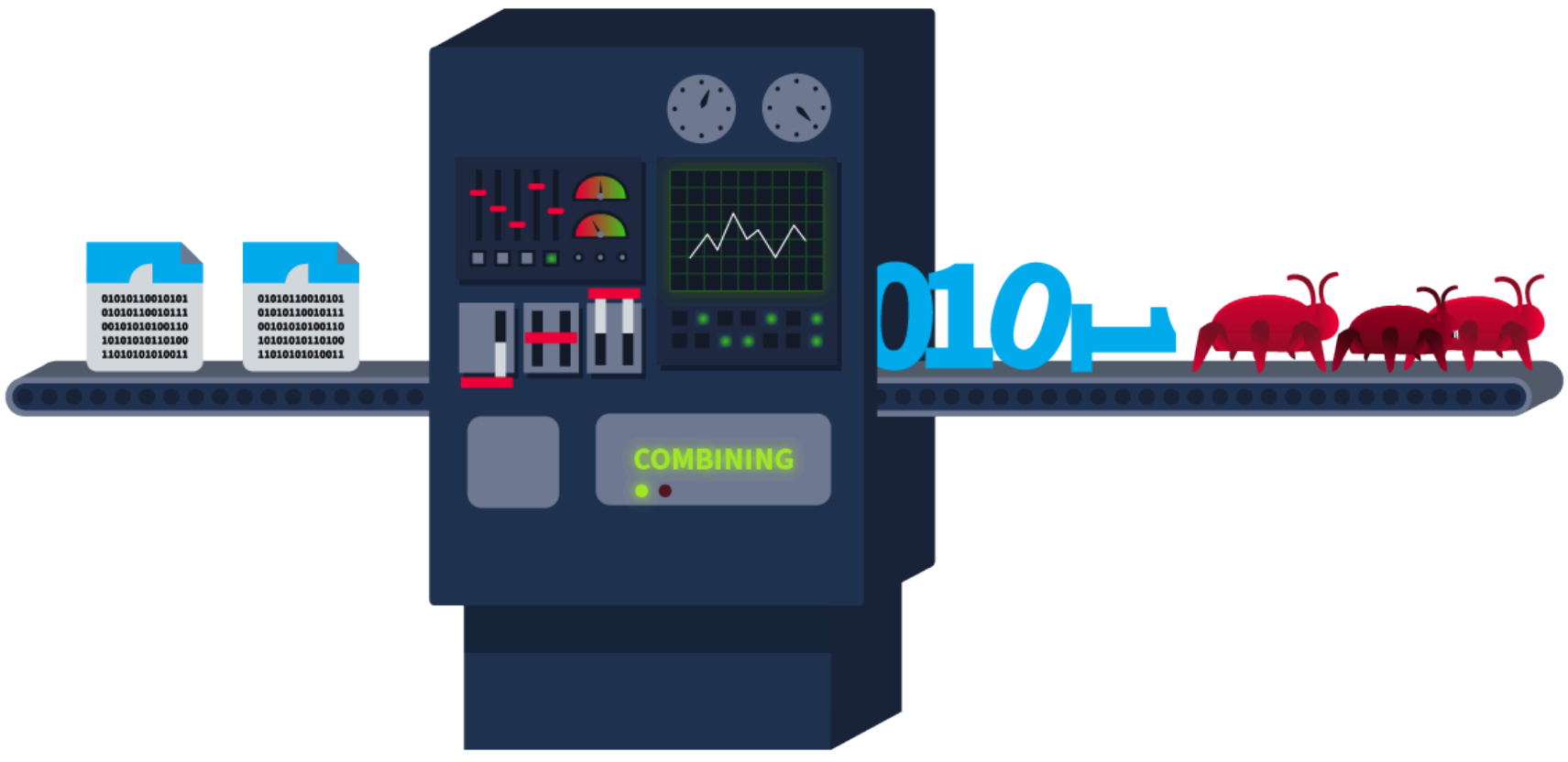
Running Snort for Detections

```
ubuntu@tryhackme:~$ sudo snort -q -l /var/log/snort -i lo -A console -c /etc/snort/sno
rt.conf07/24-10:46:52.401504  [**] [1:1000001:1] Loopback Ping Detected [**] [Priorit
y: 0] {ICMP} 127.0.0.1 -> 127.0.0.1
07/24-10:46:53.406552  [**] [1:1000001:1] Loopback Ping Detected [**] [Priority: 0] {I
CMP} 127.0.0.1 -> 127.0.0.1
```

```
07/24-10:46:54.410544  [**] [1:1000001:1] Loopback Ping Detected [**] [Priority: 0] {I
CMP} 127.0.0.1 -> 127.0.0.1
```

Running Snort on PCAP Files

We saw how Snort can be used for intrusion detection on real-time traffic. However, you may sometimes encounter a scenario where you have historical network traffic logged in a file, and you have to perform a forensic investigation to determine any signs of intrusion through that traffic. This traffic is usually logged in the standard packet capture format "PCAP". Snort is also equipped to perform detections on these PCAP files containing historical network traffic.



The following command with sudo privilege can be used to perform this action:

Running Snort onPCAP

```
ubuntu@tryhackme:~$ sudo snort -q -l /var/log/snort -r Task.pcap -A console -c /etc/sn
ort/snort.conf
```

Note: Replace the "Task.pcap" with the path to your PCAP file for analysis.